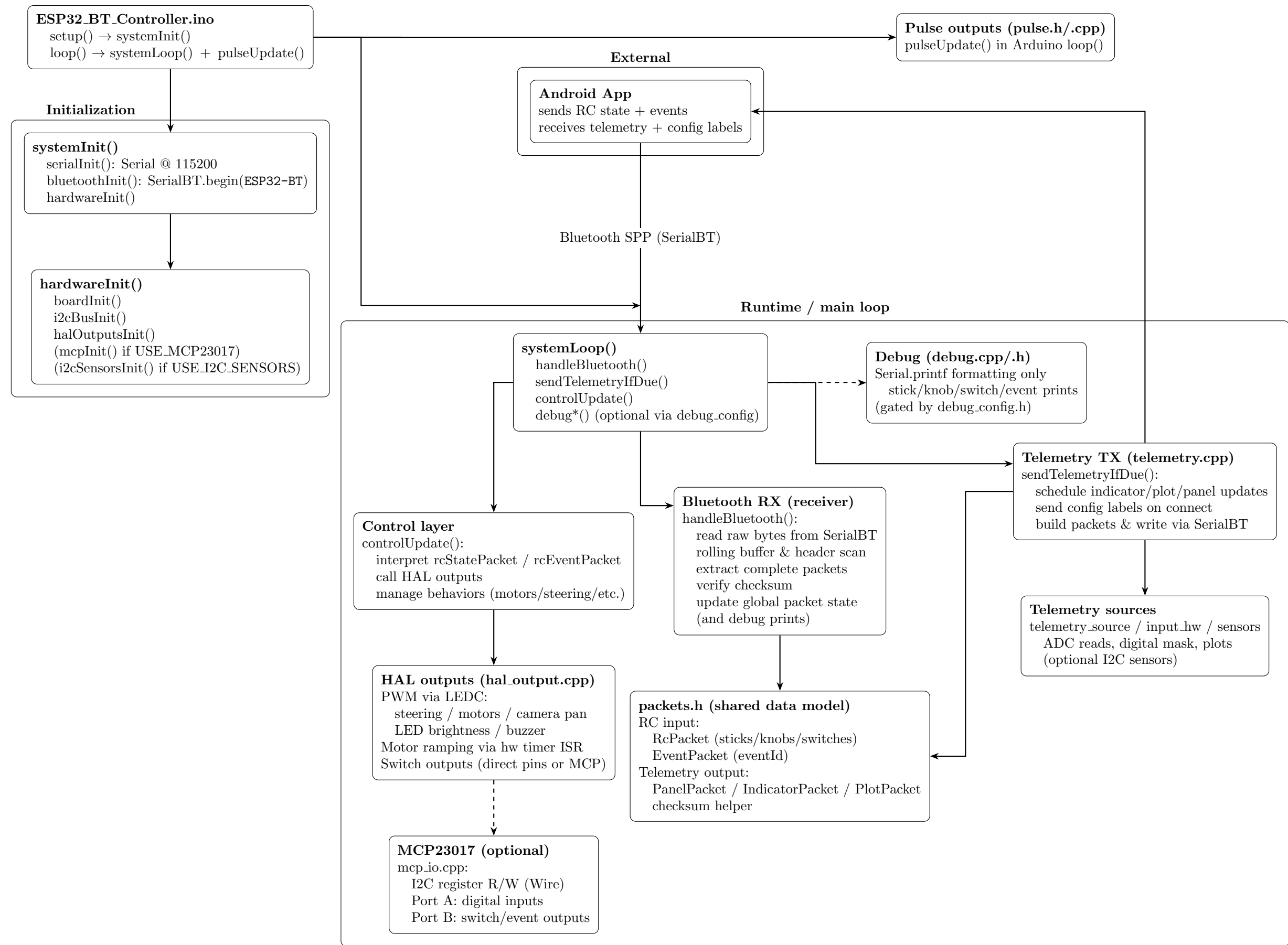




1 How the ESP32-BT Controller Works (Diagram Walkthrough)

This section explains the architecture shown in the diagram from the point of view of an Android app user. You do not need to understand the internal code to use the app, but this will help you understand what is happening when you connect, move the sticks, or look at telemetry.

1.1 Big picture

The system is a Bluetooth “remote control link”:

- The **Android App** sends **control commands** (sticks, knobs, switches, and button events) to the ESP32.
- The **ESP32 firmware** receives those commands, converts them into **hardware outputs** (PWM, motors, LED, buzzer, switches), and optionally sends **telemetry** back to the app.

The firmware is organized into two phases:

1. **Initialization** (runs once at power-on).
2. **Runtime loop** (runs continuously as long as the ESP32 is powered).

1.2 Initialization phase (left group: “Initialization”)

When the ESP32 is powered on, the Arduino entry point `setup()` calls `systemInit()`. This prepares everything needed for communication and hardware control:

`serialInit()` Starts the USB serial port at 115200. This is mainly for developer debug messages (it does not affect Bluetooth control).

`bluetoothInit()` Starts classic Bluetooth SPP using `SerialBT.begin("ESP32-BT")`. This is the Bluetooth device name your phone/app will see when scanning/pairing.

`hardwareInit()` Initializes the hardware side:

- `boardInit()` sets up board-level pins/peripherals.
- `i2cBusInit()` prepares the I²C bus (if used).
- `halOutputsInit()` prepares PWM timers / output pins.
- Optional: `mcpInit()` if an MCP23017 expander is enabled (adds more digital I/O via I²C).
- Optional: `i2cSensorsInit()` if external sensors are enabled.

1.3 Runtime phase (center group: “Runtime / main loop”)

After initialization, `loop()` runs forever. In this firmware, `loop()` calls: `systemLoop()` (core logic) and `pulseUpdate()` (momentary button pulses). This means your stick movement, switch toggles, and telemetry updates are processed continuously.

1.4 Bluetooth receive path (“Bluetooth RX”)

The Android app continuously sends small binary packets over Bluetooth.

- The function `handleBluetooth()` reads raw bytes from `SerialBT`.
- It stores the bytes in a small rolling buffer.
- It searches for known packet headers (start markers) and only processes a packet when the full length is present.
- When a valid packet is found, it is copied into shared packet structures (described below).

What the app sends There are two main command types coming from the app:

State packet (RcPacket) A “full controller state” snapshot: left stick, right stick, two knobs, and a bitmask of switch states. This is used for continuous control (steering, motor speed, pan, LED brightness, buzzer, etc.).

Event packet (EventPacket) A short “button pressed” message: contains an `eventId`. This is used for momentary actions (pulse outputs).

1.5 Shared packet model (“packets.h”)

The diagram includes `packets.h` because it is the common language between modules:

- RX writes into global packet variables (the most recent received data).
- Control reads from those packet variables to update outputs.
- Telemetry builds outgoing packets in fixed formats and writes them to Bluetooth.

In practical terms: if the app sends a new stick position, that data becomes the new “current state” used by the control logic on every loop iteration.

1.6 Control layer (“Control layer”)

The control layer is where app commands turn into physical behavior.

- `controlUpdate()` reads the latest values from the received state packet.
- It maps stick positions into PWM values.
- It calls the hardware abstraction layer (HAL) functions to apply those outputs.

Examples of what your controls do

- **Left stick X** → steering PWM.
- **Left stick Y** → left motor speed and direction.
- **Right stick X** → camera pan PWM.
- **Right stick Y** → right motor speed and direction.
- **Left knob** → LED brightness.
- **Right knob** → buzzer intensity.
- **Switches** → digital outputs (on/off).

Event buttons (momentary actions) When an event packet arrives, the receiver triggers `controlHandleEvent(eventId)`. That function converts the event ID into a short pulse on one of the outputs (for example, “press button 1” may produce a short ON pulse on a particular output pin).

1.7 HAL outputs (“hal_output.cpp”)

The HAL (hardware abstraction layer) is responsible for **writing to ESP32 hardware**:

- Uses ESP32 LEDC PWM to generate servo/motor PWM signals.
- Drives direction pins for motors (forward/reverse).
- Implements motor ramping using a hardware timer interrupt, to smooth changes in speed.
- Controls switch outputs either:
 - directly on ESP32 GPIO pins, or
 - through an optional MCP23017 I/O expander.

For the app user, this means: your UI controls directly translate into real electrical outputs on the ESP32 board.

1.8 Optional MCP23017 (“MCP23017 (optional)” box)

If enabled, the MCP23017 provides additional digital I/O over I²C. The firmware can:

- read digital inputs (e.g., indicators) from MCP Port A, and/or
- write outputs (switches/events) on MCP Port B.

1.9 Telemetry transmit path (“Telemetry TX”)

Telemetry is the opposite direction: ESP32 → Android App.

- `sendTelemetryIfDue()` runs periodically inside `systemLoop()`.
- It only sends telemetry when the phone is connected (the firmware checks `SerialBT.hasClient()`).
- On a new connection it sends a **configuration message** (labels such as plot names and panel names), so the app can automatically populate UI labels.
- It then sends periodic telemetry packets (panel, indicator, and plot values).

Telemetry sources The diagram shows a “Telemetry sources” box because telemetry does not invent values by itself. It calls helper functions such as:

- panel values (left/right),
- indicator values (analog + battery),
- plot values (three plot channels),
- optional sensor updates (if enabled).

So when you see changing telemetry on-screen, it is coming from these readings on the ESP32 side.

1.10 Debug outputs (“Debug” box)

Debug output is for development and troubleshooting:

- It prints formatted information to USB Serial.
- It can print sticks/knobs/switches/events depending on compile-time flags.
- It does not change control behavior; it only reports what is happening.

1.11 Pulse outputs (“Pulse outputs” box)

Some app controls are “momentary” (press once, output a short pulse). That behavior is handled by `pulseUpdate()`.

This ensures button presses behave like short taps rather than staying ON forever.

1.12 What to check if something doesn’t work

- **Can’t connect:** verify the device appears as ESP32-BT, pair if required, and confirm Bluetooth is enabled.
- **Controls don’t respond:** connection may be present but packets aren’t arriving; verify the app is sending state updates.
- **Telemetry missing:** telemetry only transmits when a client is connected; also verify telemetry is enabled in the build.